DAY 1 — SYSTEMVERILOG DATA TYPES

Cheat-sheet: 2-state/4-state • logic/reg/integer/time • bit/byte/shortint/int/longint • signed/unsigned • casting • X/Z • sizing

1. 2-STATE vs 4-STATE

2-state stores only 0 and 1. **4-state** stores 0, 1, X and Z. In RTL simulation, 4-state types help expose unknown/uninitialized behavior.

2-state	4-state
bit	logic
byte	reg
shortint	integer
int	time
longint	

Memory trick: 2-state = “clean 0/1 data”; 4-state = “realistic simulation values including unknown/high-Z.”

2. COMMON BUILT-IN TYPES

Type	Width	State	Default signedness / note
bit	1	2	unsigned
byte	8	2	signed
shortint	16	2	signed
int	32	2	signed
longint	64	2	signed
logic	user-sized	4	unsigned unless signed is specified
reg	user-sized	4	legacy Verilog variable type
integer	32	4	signed
time	64	4	unsigned; simulation time

3. logic vs reg

Both are 4-state. **logic** is the modern SystemVerilog choice and is commonly used for RTL signals. **reg** is the older Verilog keyword. Do not confuse logic with a 2-state type.

```
logic [7:0] data; // 8-bit, 4-state
reg [7:0] old_data; // legacy style
```

4. integer and time

integer: 32-bit, signed, 4-state; frequently useful in testbenches/loop control. **time**: 64-bit, unsigned, 4-state; represents simulation time.

```
integer i; time t;
i = 10; t = $time;
```

5. X and Z

X = unknown — simulator cannot determine 0 or 1. **Z = high impedance** — commonly associated with tri-state/floating signals.

```
logic a; a = 1'bx; // X
logic b; b = 1'bz; // Z
```

Important: 2-state variables cannot retain X/Z; conversion to a 2-state type loses that information.

DAY 1 — SIGNEDNESS, CASTING & SIZING

6. SIGNED vs UNSIGNED

Signedness changes how a bit pattern is interpreted in arithmetic and comparisons. For n bits: unsigned range = 0 to 2^n–1; signed two's-complement range = -2^{n-1} to 2^{n-1}–1.

```
logic [7:0] u; // 8-bit unsigned: 0..255
logic signed [7:0] s; // 8-bit signed: -128..127
```

Key example: 8'b1111_1111 = 255 when unsigned, but –1 when interpreted as signed.

7. IMPLICIT CASTING

Implicit conversion happens automatically when operands/assignments require compatible types or widths. It can introduce extension, truncation, or signedness effects.

```
byte b; int a; a = b; // automatic conversion
```

Rule: never assume the result is what you intended when widths/signedness differ—check the expression and destination type.

8. EXPLICIT CASTING

Explicit casting makes the desired conversion clear. Common forms include a type cast, sized cast, or signedness cast.

```
int'(value) // cast to int
byte'(value) // cast to byte
signed'(value) // signed interpretation
unsigned'(value) // unsigned interpretation
8'(value) // sized casting
```

Best practice: use explicit casts when crossing widths, signed/unsigned boundaries, or interface/data-format boundaries.

9. SIZING OF LITERALS

A sized literal uses **size'base_value**. Base letters: **b**=binary, **o**=octal, **d**=decimal, **h**=hexadecimal.

```
8'b1010_0101 // 8-bit binary
8'hA5 // 8-bit hexadecimal
8'd25 // 8-bit decimal
12'o777 // 12-bit octal
Underscores improve readability: 32'hDEAD_BEEF
```

10. WIDTH: TRUNCATION & EXTENSION

Truncation: wider value → narrower destination; upper bits can be discarded.

```
logic [11:0] a = 12'hACF; logic [7:0] b; b = a; // upper 4 bits lost
```

Extension: narrower value → wider destination. Unsigned values are generally zero-extended; signed values are sign-extended when the expression is treated as signed.

```
8'b0010_1010 → 12'b0000_0010_1010 // zero extension
8'b1111_1010 (signed) → 12'b1111_1111_1010 // sign extension
```

11. DAY 1 RAPID REVISION

Question	Answer
How many states in bit?	2: 0/1
How many states in logic?	4: 0/1/X/Z
Modern RTL type?	logic
Legacy Verilog type?	reg
integer?	32-bit, signed, 4-state
time?	64-bit, unsigned, 4-state
int default?	32-bit signed, 2-state
What is X?	Unknown
What is Z?	High impedance
Why signed?	Changes numerical interpretation/arithmetic
Why sizing?	Controls width; prevents surprises
Why explicit cast?	Makes conversion intent clear